1. Initial iptables

```
Activities  Terminal ▾                                   Sun 15:03
                                                student@student-virtual-machine: ~
File  Edit  View  Search  Terminal  Help
student@student-virtual-machine:~$ sudo iptables -L
Chain INPUT (policy ACCEPT)
target     prot opt source               destination

Chain FORWARD (policy ACCEPT)
target     prot opt source               destination

Chain OUTPUT (policy ACCEPT)
target     prot opt source               destination
student@student-virtual-machine:~$ sudo iptables -t nat -L
Chain PREROUTING (policy ACCEPT)
target     prot opt source               destination

Chain INPUT (policy ACCEPT)
target     prot opt source               destination

Chain OUTPUT (policy ACCEPT)
target     prot opt source               destination

Chain POSTROUTING (policy ACCEPT)
target     prot opt source               destination
student@student-virtual-machine:~$
```

2. Dropping ICMP messages.

```
                                                root@kali: ~
File  Edit  View  Search  Terminal  Help
root@kali:~# ping -c 4 machineA
PING machineA (192.168.1.100) 56(84) bytes of data.
64 bytes from machineA (192.168.1.100): icmp_seq=1 ttl=64 time=0.324 ms
64 bytes from machineA (192.168.1.100): icmp_seq=2 ttl=64 time=0.359 ms
64 bytes from machineA (192.168.1.100): icmp_seq=3 ttl=64 time=0.350 ms
64 bytes from machineA (192.168.1.100): icmp_seq=4 ttl=64 time=0.368 ms

--- machineA ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3056ms
rtt min/avg/max/mdev = 0.324/0.350/0.368/0.016 ms
root@kali:~#
```

Using this command I blocked incoming ICMP messages: *iptables -A INPUT -p icmp -j DROP*

```
student@student-virtual-machine:~$ sudo iptables -L
Chain INPUT (policy ACCEPT)
target     prot opt source               destination
DROP       icmp --  anywhere             anywhere

Chain FORWARD (policy ACCEPT)
target     prot opt source               destination

Chain OUTPUT (policy ACCEPT)
target     prot opt source               destination
```

Machine B does not get replay

```
root@kali: ~
File  Edit  View  Search  Terminal  Help
root@kali:~# ping -c 4 machineA
PING machineA (192.168.1.100) 56(84) bytes of data.
64 bytes from machineA (192.168.1.100): icmp_seq=1 ttl=64 time=0.324 ms
64 bytes from machineA (192.168.1.100): icmp_seq=2 ttl=64 time=0.359 ms
64 bytes from machineA (192.168.1.100): icmp_seq=3 ttl=64 time=0.350 ms
64 bytes from machineA (192.168.1.100): icmp_seq=4 ttl=64 time=0.368 ms

--- machineA ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3056ms
rtt min/avg/max/mdev = 0.324/0.350/0.368/0.016 ms
root@kali:~# ping -c 4 machineA
PING machineA (192.168.1.100) 56(84) bytes of data.

--- machineA ping statistics ---
4 packets transmitted, 0 received, 100% packet loss, time 3066ms

root@kali:~#
```

3. Firewall rules on SSH traffic

Part 3A: rejects all traffic and then only allow ssh traffic on port 22.

```
File  Edit  View  Search  Terminal  Help
student@student-virtual-machine:~$ sudo iptables -A INPUT -s machineB -j DROP
student@student-virtual-machine:~$ sudo iptables -A INPUT -p tcp --dport 22 -j ACCEPT
student@student-virtual-machine:~$ sudo iptables -L
Chain INPUT (policy ACCEPT)
target     prot opt source               destination
DROP       all  --  machineB             anywhere
ACCEPT     tcp  --  anywhere             anywhere             tcp dpt:ssh

Chain FORWARD (policy ACCEPT)
target     prot opt source               destination

Chain OUTPUT (policy ACCEPT)
target     prot opt source               destination
student@student-virtual-machine:~$
```

```
Applications ▾   Places ▾   Terminal ▾                Sun 15:59 ●
root@kali: ~
File  Edit  View  Search  Terminal  Help
root@kali:~# ping -c 4 machineA
PING machineA (192.168.1.100) 56(84) bytes of data.
^[[B
--- machineA ping statistics ---
4 packets transmitted, 0 received, 100% packet loss, time 3068ms

root@kali:~# ssh student@machineA
```

Part 3B: allow only ssh traffic on port 22 and then drops all the traffic

```
File  Edit  View  Search  Terminal  Help
student@student-virtual-machine:~$ sudo iptables -A INPUT -p tcp --dport 22 -j ACCEPT
student@student-virtual-machine:~$ sudo iptables -A INPUT -s machineB -j DROP
student@student-virtual-machine:~$ sudo iptables -L
Chain INPUT (policy ACCEPT)
target     prot opt source               destination
ACCEPT     tcp  --  anywhere             anywhere             tcp dpt:ssh
DROP       all  --  machineB             anywhere

Chain FORWARD (policy ACCEPT)
target     prot opt source               destination

Chain OUTPUT (policy ACCEPT)
target     prot opt source               destination
student@student-virtual-machine:~$
```

```
student@student-virtual-machine: ~
File  Edit  View  Search  Terminal  Help
root@kali:~# ping -c 4 machineA
PING machineA (192.168.1.100) 56(84) bytes of data.

--- machineA ping statistics ---
4 packets transmitted, 0 received, 100% packet loss, time 3051ms

root@kali:~# ssh student@machineA
The authenticity of host 'machinea (192.168.1.100)' can't be established.
ECDSA key fingerprint is SHA256:Tbkbp8Zx2fzrP0JiEfpo99BpG9oZh9/uTgZsbKwrGHA.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added 'machinea,192.168.1.100' (ECDSA) to the list of known hosts.
student@machinea's password:
Welcome to Ubuntu 18.04.2 LTS (GNU/Linux 4.15.0-55-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:     https://landscape.canonical.com
 * Support:        https://ubuntu.com/advantage


 * Canonical Livepatch is available for installation.
   - Reduce system reboots and improve kernel security. Activate at:
     https://ubuntu.com/livepatch

539 packages can be updated.
445 updates are security updates.

New release '20.04.6 LTS' available.
Run 'do-release-upgrade' to upgrade to it.


The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

student@student-virtual-machine:~$ whoami
student
student@student-virtual-machine:~$
```

4. The rule will drop all TCP packets that do not have the SYN flags set. This rule triggered before all the rules we see. The table and chan that this rule is in is the mangle table, PREROUTING Chain.

```
student@student-virtual-machine:~$ sudo iptables -t mangle -L
Chain PREROUTING (policy ACCEPT)
target     prot opt source               destination
DROP       tcp  --  anywhere             anywhere             tcp flags:SYN/NONE

Chain INPUT (policy ACCEPT)
target     prot opt source               destination

Chain FORWARD (policy ACCEPT)
target     prot opt source               destination

Chain OUTPUT (policy ACCEPT)
target     prot opt source               destination

Chain POSTROUTING (policy ACCEPT)
target     prot opt source               destination
student@student-virtual-machine:~$
```

5. The complete command is *sudo iptables -A INPUT -p icmp -m mac --mac-source 00:50:56:b0:30:b8 -j ACCEPT.* However, after that you have to use the command, sudo iptables -A INPUT -p icmp -j DROP to drop all the ICMP messages from any other sources.

```
Activities   Terminal                                   Sun 17:29
                                                student@student-virtual-machine: ~
File  Edit  View  Search  Terminal  Help
student@student-virtual-machine:~$ sudo iptables -L
Chain INPUT (policy ACCEPT)
target     prot opt source               destination
ACCEPT     icmp --  anywhere             anywhere             MAC 00:50:56:B0:30:B8
DROP       icmp --  anywhere             anywhere

Chain FORWARD (policy ACCEPT)
target     prot opt source               destination

Chain OUTPUT (policy ACCEPT)
target     prot opt source               destination
student@student-virtual-machine:~$
```

6. The firewall rules got erased after rebooting. Then, I saved the firewall rules to the iptables.rules. After I rebooted the machine, I was able to restore the firewall rules with the file.

```
File  Edit  View  Search  Terminal  Help
student@student-virtual-machine:~$ ls
Desktop  Documents  Downloads  iptables.rules  Music  Pictures  Public  Templat
es  Videos
student@student-virtual-machine:~$ sudo iptables -L
[sudo] password for student:
Chain INPUT (policy ACCEPT)
target     prot opt source               destination

Chain FORWARD (policy ACCEPT)
target     prot opt source               destination

Chain OUTPUT (policy ACCEPT)
target     prot opt source               destination
student@student-virtual-machine:~$ sudo iptables-restore < ./iptables.rules
student@student-virtual-machine:~$ sudo iptables -L
Chain INPUT (policy ACCEPT)
target     prot opt source               destination
DROP       icmp --  anywhere             anywhere

Chain FORWARD (policy ACCEPT)
target     prot opt source               destination

Chain OUTPUT (policy ACCEPT)
target     prot opt source               destination
student@student-virtual-machine:~$
```

PART II

2. The name of the dante client is supposed to be dante.conf and for the server it's supposed to be danted.conf.

```
File  Edit  View  Search  Terminal  Help
  GNU nano 2.9.3                    /etc/dante.conf

# $Id: socks.conf,v 1.28 2001/12/12 13:56:42 karls Exp $
#
# A sample dante.conf

# The configfile is divided into two parts; first misc. settings,
# then the routes.  Objects in '[]' are optional.
#
#
```

```
File  Edit  View  Search  Terminal  Help
  GNU nano 2.9.3                    /etc/danted.conf

# $Id: sockd.conf,v 1.43 2005/12/26 16:35:26 michaels Exp $
#
# A sample danted.conf
#
#
# The configfile is divided into three parts;
```

3. There was an error with the external traffic name in the configuration file.

```
student@student-virtual-machine:~/Downloads$ sudo systemctl status danted.service
● danted.service - SOCKS (v4 and v5) proxy daemon (danted)
   Loaded: loaded (/lib/systemd/system/danted.service; disabled; vendor preset: enabled)
   Active: failed (Result: exit-code) since Sun 2023-10-15 18:52:35 EDT; 35s ago
     Docs: man:danted(8)
           man:danted.conf(5)
  Process: 6139 ExecStart=/usr/sbin/danted -D (code=exited, status=1/FAILURE)
  Process: 6135 ExecStartPre=/bin/sh -c         uid=`sed -n -e "s/[[:space:]]//g" -e "s/#.*//" -e "/^user\.privileged/{s/[^:]*://p;q;}" /etc/danted.conf`;      if [ -n "$uid" ]; then

Oct 15 18:52:35 student-virtual-machine systemd[1]: Starting SOCKS (v4 and v5) proxy daemon (danted)...
Oct 15 18:52:35 student-virtual-machine danted[6139]: Oct 15 18:52:35 (1697410355.541363) danted[6139]: error: /etc/danted.conf: problem on line 60 near token "wlan0": could not resolv
Oct 15 18:52:35 student-virtual-machine danted[6139]: Oct 15 18:52:35 (1697410355.541715) danted[6139]: alert: mother[1/1]: shutting down
Oct 15 18:52:35 student-virtual-machine systemd[1]: danted.service: Control process exited, code=exited status=1
Oct 15 18:52:35 student-virtual-machine systemd[1]: danted.service: Failed with result 'exit-code'.
Oct 15 18:52:35 student-virtual-machine systemd[1]: Failed to start SOCKS (v4 and v5) proxy daemon (danted).
student@student-virtual-machine:~/Downloads$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
       valid_lft forever preferred_lft forever
2: ens160: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 00:50:56:b0:8b:e0 brd ff:ff:ff:ff:ff:ff
    inet 192.168.1.100/24 brd 192.168.1.255 scope global dynamic noprefixroute ens160
       valid_lft 4751sec preferred_lft 4751sec
    inet6 fe80::986d:3104:de42:eda6/64 scope link noprefixroute
       valid_lft forever preferred_lft forever
student@student-virtual-machine:~/Downloads$
```

Replaced the wlan0 with ens160 and that fixed the problem.

```
student@student-virtual-machine:~/Downloads$ sudo /etc/init.d/danted start
[ ok ] Starting danted (via systemctl): danted.service.
student@student-virtual-machine:~/Downloads$
```

The problem was that the external traffic was set to wlan0 and the machine is connected to the router via ethernet, so I had to set the wlan0 to ens160 for external traffic.

```
student@student-virtual-machine:~/Downloads$ sudo /etc/init.d/danted stop
[ ok ] Stopping danted (via systemctl): danted.service.
student@student-virtual-machine:~/Downloads$
```

4. The debug flag is -d. You can set the level of debug from 0 - 2. -d 0 for no debug logging, -d 1 for some debug logging, -d 2 for verbose debug logging.

5. When I start the proxied client with “socksify firefox”, I see that the firefox browser opens up. In the debug mode in the terminal it’s also showing the following:

```
File  Edit  View  Search  Terminal  Help
Oct 15 19:31:36 (1697412696.700464) danted[6778]: debug: socks_freeinbuffer(): fd 18, which 1, free: 65536
Oct 15 19:31:36 (1697412696.700466) danted[6778]: debug: socks_recvfrom(): fd 19, len 65536, flags 0
Oct 15 19:31:36 (1697412696.700469) danted[6778]: debug: socks_getfrombuffer(): fd = 19, get up to 65536 decoded bytes from read buffer which currently has 0 decoded, 0 encoded.  Flags
 = 0
Oct 15 19:31:36 (1697412696.700471) danted[6778]: debug: socks_freeinbuffer(): fd 19, which 0, free: 65536
Oct 15 19:31:36 (1697412696.700477) danted[6778]: debug: socks_recvfrom(): read 0/65536 bytes from socket, fd 19, errno = 0 (no system error)
Oct 15 19:31:36 (1697412696.700480) danted[6778]: debug: io_tcp_rw(): got EOF on in->s (fd 19) when trying to read 65536 bytes.  Status of out (fd 18): fin: 0
Oct 15 19:31:36 (1697412696.700482) danted[6778]: debug: io_tcp_rw(): shutting down out->s (fd 18) for writing
Oct 15 19:31:36 (1697412696.700501) danted[6778]: debug: io_update(): bwused 0, bw_shmid 0, mstats_shmid 0
Oct 15 19:31:36 (1697412696.700503) danted[6778]: debug: doto_tcp(): bwused = 0
Oct 15 19:31:36 (1697412696.700507) danted[6778]: debug: run_io(): first select; readable/connected?
Oct 15 19:31:36 (1697412696.700509) danted[6778]: debug: io_gettimeout(): last_time = 4424, tnow = 4424, last_timeout_isset = 0, last_timeout = 1.000000
Oct 15 19:31:36 (1697412696.700511) danted[6778]: debug: io_allocated(): iov #0 allocated for tcp
Oct 15 19:31:36 (1697412696.700512) danted[6778]: debug: io_allocated(): iov #1 allocated for tcp
Oct 15 19:31:36 (1697412696.700514) danted[6778]: debug: io_allocated(): iov #2 allocated for tcp
Oct 15 19:31:36 (1697412696.700516) danted[6778]: debug: io_allocated(): iov #3 allocated for tcp
Oct 15 19:31:36 (1697412696.700517) danted[6778]: debug: io_allocated(): iov #4 allocated for tcp
Oct 15 19:31:36 (1697412696.700518) danted[6778]: debug: io_allocated(): iov #5 allocated for tcp
Oct 15 19:31:36 (1697412696.700520) danted[6778]: debug: io_allocated(): iov #6 allocated for tcp
Oct 15 19:31:36 (1697412696.700521) danted[6778]: debug: io_allocated(): iov #7 allocated for tcp
Oct 15 19:31:36 (1697412696.700523) danted[6778]: debug: io_allocated(): iov #8 allocated for tcp
Oct 15 19:31:36 (1697412696.700524) danted[6778]: debug: io_allocated(): iov #9 allocated for tcp
Oct 15 19:31:36 (1697412696.700526) danted[6778]: debug: io_allocated(): iov #10 allocated for tcp
Oct 15 19:31:36 (1697412696.700527) danted[6778]: debug: io_allocated(): allocated for tcp: 11, udp: 0
Oct 15 19:31:36 (1697412696.700529) danted[6778]: debug: io_gettimeout(): timeout for iov #0 is in -1s
Oct 15 19:31:36 (1697412696.700531) danted[6778]: debug: io_gettimeout(): timeout for iov #1 is in -1s
Oct 15 19:31:36 (1697412696.700533) danted[6778]: debug: io_gettimeout(): timeout for iov #2 is in -1s
Oct 15 19:31:36 (1697412696.700534) danted[6778]: debug: io_gettimeout(): timeout for iov #3 is in -1s
Oct 15 19:31:36 (1697412696.700536) danted[6778]: debug: io_gettimeout(): timeout for iov #4 is in -1s
Oct 15 19:31:36 (1697412696.700537) danted[6778]: debug: io_gettimeout(): timeout for iov #5 is in -1s
Oct 15 19:31:36 (1697412696.700539) danted[6778]: debug: io_gettimeout(): timeout for iov #6 is in -1s
Oct 15 19:31:36 (1697412696.700540) danted[6778]: debug: io_gettimeout(): timeout for iov #7 is in -1s
Oct 15 19:31:36 (1697412696.700542) danted[6778]: debug: io_gettimeout(): timeout for iov #8 is in -1s
Oct 15 19:31:36 (1697412696.700544) danted[6778]: debug: io_gettimeout(): timeout for iov #9 is in -1s
Oct 15 19:31:36 (1697412696.700545) danted[6778]: debug: io_gettimeout(): timeout for iov #10 is in -1s
Oct 15 19:31:36 (1697412696.700575) danted[6778]: debug: pre-select: nfds = 31, rset = 0x556972143e70 (7, 8, 10, 11, 12, 13, 14, 16, 18, 20, 21, 22, 24, 26, 27, 28, 30, ), bufrset = 0x
55697212ee30 (10, 11, 12, 13, 14, 16, 18, 20, 21, 22, 24, 26, 27, 28, 30, ), buffwset = 0x55697212ec20 (10, 11, 12, 13, 14, 16, 18, 20, 21, 22, 24, 26, 27, 28, 30, ), wset = 0x55697214
44a0 (), xset = 0x5569721446b0 (), timeout = 0x0
```

6. After stopping and starting it back up. I see that there is traffic appearing in the wireshark captured traffic. The reason is that the client is connecting to the Dante proxy server through the localhost interface and then proxy server is connecting to the destination server through the eth0, or in my case ens160, interface. The thing is that wireshark captures traffic right before it goes out the interface and that's why I am able to see the traffic from the Dante server. The proxy server is between the client and the wireshark.

Client → Dante → Wireshark → eth0(ens160) → [internet] → Destination server

7. The 0.0.0.0/0 to: 0.0.0.0/0 via: 127.0.0.1 port = 1080. Basically connecting to the localhost interface port 1080.

```
#=========================Avi's changes
route {
        from: 0.0.0.0/0   to: 0.0.0.0/0   via: 127.0.0.1 port = 1080
        protocol: tcp udp                # server supports tcp and udp.
        proxyprotocol: socks_v4 socks_v5 # server supports socks v4 and v5.
        method: none #username     # we are willing to authenticate
                                  # via method "none", not "username".
}
route {
        from: 0.0.0.0/0   to: . via: 127.0.0.1 port = 1080
        protocol: tcp udp                # server supports tcp and udp.
        proxyprotocol: socks_v4 socks_v5 # server supports socks v4 and v5.
        method: none #username     # we are willing to authenticate
                                  # via method "none", not "username".
}
```